

Member

고영후, 서울 스포츠사이언스학부,

김한준, 에리카 로봇공학과, hanjun0126@hanyang.ac.kr

1. Proposal(Option A)

이제 우리는 생성형 인공지능을 쉽게 접할 수 있습니다. 그런데 이러한 인공지능이 어떻게 작동하는지는 잘 모르는 경우가 많습니다. 그 중 트랜스포머는 이런 인공지능의 패러다임을 크게 바꿨고, 여러 모델로 확장되었습니다. 또 VAE는 데이터를 잠재공간으로 인코딩하고 다시 디코딩하는 과정을 통해 데이터의 확률분포를 학습합니다. 저희는 이미지가 어떻게 생성되는지 알아보고자 이 주제를 선택하였습니다. 트랜스포머와 많은 데이터를 학습하는것은 환경에 제한이 있기 때문에, 간단한 데이터 셋인 채널 수도 1개이고 잠재 공간을 크게 안써도 되는 MNIST를 사용하여 숫자 이미지 생성을 하는 것을 목표로 하였습니다. 간단하지만 이번 프로젝트를 통해서 인공지능 모델이 이미지 생성을 어떻게 할 수 있는지를 알아내고, 우리는 데이터의 무엇을 학습하는지를 확인하고자 합니다.

2. Dataset

프로젝트에 사용되는 MNIST 데이터셋은 아래와 같이 사용할 수 있습니다.

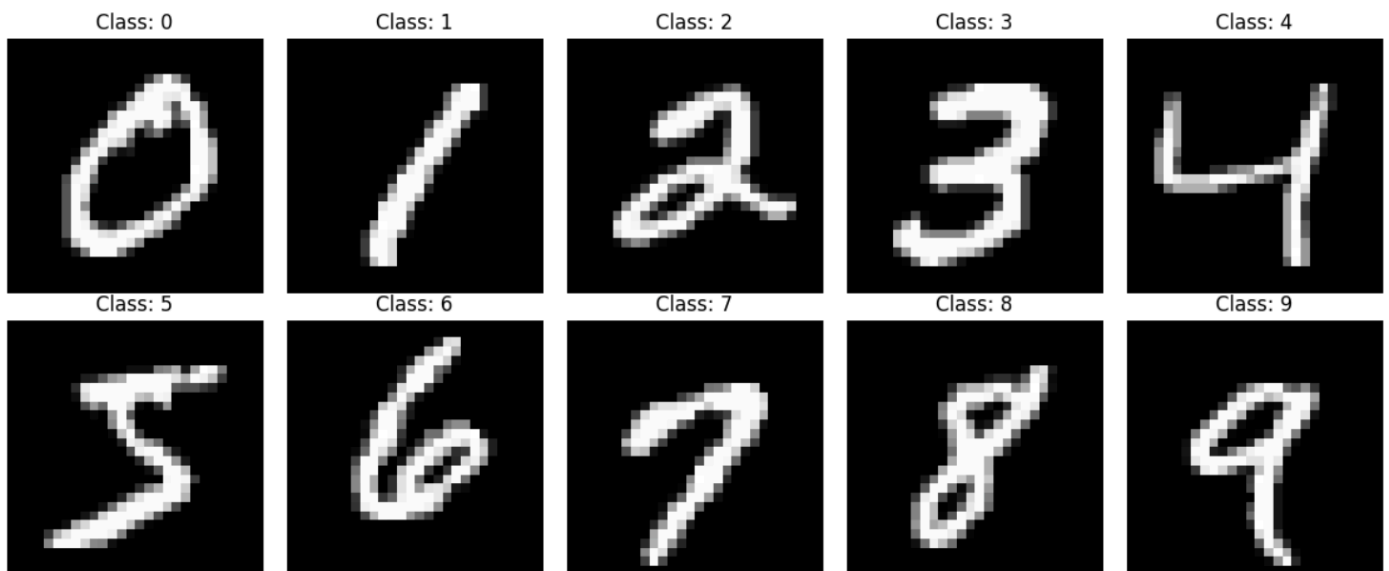
```
1 import matplotlib.pyplot as plt
2 from torchvision import datasets, transforms
3
4 # 1. MNIST 데이터셋 불러오기
5 mnist_train = datasets.MNIST(
6     root='./data',
7     train=True,
8     download=True,
9     transform=transforms.ToTensor()
10 )
11
12 # 2. 0부터 9까지 각 클래스별로 하나의 이미지를 저장할 딕셔너리
13 class_images = {}
14
15 # 데이터셋을 하나씩 확인하며 각 클래스의 첫 번째 이미지를 수집
16 for image, label in mnist_train:
17     if label not in class_images:
```

```

18         # [1, 28, 28] 텐서를 [28, 28] 넘파이 배열로 변환하여 저장
19         class_images[label] = image.squeeze().numpy()
20
21     # 0부터 9까지 10개의 클래스를 모두 찾았다면 탐색 종료
22     if len(class_images) == 10:
23         break
24
25 # 3. 수집한 이미지를 0부터 9까지 순서대로 시각화
26 fig, axes = plt.subplots(2, 5, figsize=(12, 5))
27
28 for i in range(10):
29     row = i // 5 # 행 위치 (0 또는 1)
30     col = i % 5  # 열 위치 (0부터 4)
31
32     axes[row, col].imshow(class_images[i], cmap='gray')
33     axes[row, col].set_title(f"Class: {i}")
34     axes[row, col].axis('off')
35
36 plt.tight_layout()
37 plt.show()

```

데이터는 1개의 채널을 가진 가로와 세로 모두 28의 크기를 가진 0부터 9까지의 손글씨 숫자 이미지입니다. 각 클래스별로 이미지를 확인하면 아래와 같습니다.



VAE 구현하기

필요한 라이브러리를 먼저 불러오고, 시드를 설정합니다.

```

1 import numpy as np

```

```

2 import math, os, random, time
3 import matplotlib.pyplot as plt
4
5 import torch
6 import torch.nn as nn
7 import torch.nn.functional as F
8 from torch import optim
9 from torch.utils.data import DataLoader, Dataset, Subset
10 from torchvision import datasets, transforms, utils as vutils
11
12
13 def set_random_seed(seed_value):
14     np.random.seed(seed_value)
15     torch.manual_seed(seed_value)
16     torch.cuda.manual_seed_all(seed_value)
17     torch.backends.cudnn.deterministic = True
18     torch.backends.cudnn.benchmark = False
19
20 seed_value = 42
21 set_random_seed(seed_value)
22
23 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
24 print("device:", device)

```

학습에 사용될 MNIST 데이터셋을 불러옵니다.

```

1 class IndexedMNIST(datasets.MNIST):
2     def __getitem__(self, index):
3         img, target = super().__getitem__(index)
4         return img, target, index # (x, y, idx)
5
6 # — 1. 전체 데이터셋 로드 —
7 transform = transforms.Compose([
8     transforms.ToTensor(),
9     transforms.Lambda(lambda x: (x > 0.5).float())
10 ])
11
12 # 먼저 전체 train/test 데이터셋을 로드합니다.
13 train_set_full = IndexedMNIST(root='./data', train=True,
14                               transform=transform, download=True)
15 test_set_full = IndexedMNIST(root='./data', train=False,
16                              transform=transform, download=True)
17
18 digit=3 # MNIST에서 숫자 하나만 골라오겠습니다.

```

```

17 train_indices = (train_set_full.targets == digit).nonzero().squeeze()
18 test_indices = (test_set_full.targets == digit).nonzero().squeeze()
19
20 train_set = Subset(train_set_full, train_indices)
21 test_set = Subset(test_set_full, test_indices)
22
23 BATCH = 128
24 train_loader = DataLoader(train_set, batch_size=BATCH, shuffle=True,
    num_workers=2, pin_memory=True)
25 test_loader = DataLoader(test_set, batch_size=BATCH, shuffle=False,
    num_workers=2, pin_memory=True)
26
27 N_TRAIN = len(train_set)
28 IMG_SHAPE = (1, 28, 28)
29 D = 28*28
30 print("N_TRAIN:", N_TRAIN)

```

VAE를 정의하겠습니다.

```

1 # Vanilla VAE
2 # Encoder network:  $x \rightarrow z$ 
3 class Encoder(nn.Module):
4     def __init__(self, zdim=20, hidden=400):
5         super().__init__()
6         self.f = nn.Sequential(
7             nn.Flatten(),
8             nn.Linear(D, hidden), nn.ReLU(True),
9             nn.Linear(hidden, hidden), nn.ReLU(True),
10        )
11        self.mu = nn.Linear(hidden, zdim)
12        self.logvar = nn.Linear(hidden, zdim)
13
14        def forward(self, x):
15            h = self.f(x)
16            return self.mu(h), self.logvar(h)
17
18 # Decoder network:  $z \rightarrow x$ 
19 class Decoder(nn.Module):
20     def __init__(self, zdim=20, hidden=400):
21         super().__init__()
22        self.net = nn.Sequential(
23            nn.Linear(zdim, hidden), nn.ReLU(True),
24            nn.Linear(hidden, hidden), nn.ReLU(True),
25            nn.Linear(hidden, D) # logits

```

```

26         )
27
28     def forward(self, z):
29         logits = self.net(z)
30         return logits.view(-1, *IMG_SHAPE) # Bx1x28x28
31
32 def bernoulli_nll_logits(x, logits, reduction='sum'):
33     # BCE with logits:  $-\log p(x|z)$  for Bernoulli
34     #  $x \in \{0,1\}$ ; logits  $\in \mathbb{R}$ 
35     loss = F.binary_cross_entropy_with_logits(logits, x,
36     reduction=reduction)
37     return loss
38
39 class VanillaVAE(nn.Module):
40     def __init__(self, zdim=20, hidden=400, beta=1.0):
41         super().__init__()
42         self.enc = Encoder(zdim, hidden)
43         self.dec = Decoder(zdim, hidden)
44         self.beta = beta
45         self.zdim = zdim
46
47     @staticmethod
48     def reparam(mu, logvar):
49         # TODO 1: reparameterization trick을 구현합니다.
50         ### Implement Here ###
51         std = torch.exp(0.5 * logvar)
52         eps = torch.randn_like(std)
53         return mu + eps*std
54
55     def forward(self, x):
56         mu, logvar = self.enc(x)
57         z = self.reparam(mu, logvar)
58         x_logits = self.dec(z)
59         recon = bernoulli_nll_logits(x, x_logits, reduction='sum')
60         kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
61         nelbo = recon + self.beta * kl
62         B = x.size(0)
63         return {"x_logits": x_logits, "nelbo": nelbo, "recon": recon,
64         "kl": kl, "B": B}
65
66 def train_vanilla_vae(model, loader, epochs=10, lr=1e-3, beta=0.1):
67     model.to(device)
68     opt = optim.Adam(model.parameters(), lr=lr)
69     history = []

```

```

68     for epoch in range(1, epochs+1):
69         model.train()
70
71         model.beta = beta
72         s_nelbo = s_recon = s_kl = n = 0
73         for x, _, _ in loader:
74             x = x.to(device)
75             out = model(x)
76             opt.zero_grad()
77             out["nelbo"].backward()
78             opt.step()
79             s_nelbo += out["nelbo"].item()
80             s_recon += out["recon"].item()
81             s_kl     += out["kl"].item()
82             n        += out["B"]
83         history.append({
84             "epoch": epoch,
85             "nelbo_per_data": s_nelbo/n,
86             "recon_per_data": s_recon/n,
87             "kl_per_data": s_kl/n,
88             "beta": model.beta
89         })
90         print(f"[Vanilla] ep={epoch:03d}  $\beta$ ={model.beta:.3f} "
91             f"NELBO={s_nelbo/n:.3f} Recon={s_recon/n:.3f} KL=
92             {s_kl/n:.3f}")
92     return history

```